

PEBLO

Take-Home Challenge

Full Stack Developer

Build a Collaborative AI Notes Workspace

A practical, end-to-end assignment to evaluate how you design and ship full-stack products — from data model to interface, with AI woven in meaningfully.

ESTIMATED EFFORT Self-paced · scope to your available time

DELIVERABLES GitHub repo · README · demo video · sample outputs

STACK Your choice — pick what you know best

About Peblo

Peblo is a **seed-funded, stealth-mode ed-tech startup**, backed by the **Eleven Group** — a 35-year-old diversified business group headquartered in Gurgaon. We are building India's AI-powered learning universe for children: a single platform where kids can watch original Indian animated stories, play games, take quizzes, explore the news, and grow alongside their very own AI buddy.

We care deeply about craft, and we'd love for you to get a feel for what we're building before we go further. Here's the *Peblo Universe Song* that walks through what we're planning to offer in the app and how we approach it creatively:

[Watch the Peblo Universe Song →](https://youtu.be/heubljqtJ0I?si=_WCKXqQx_xXpj5th)

https://youtu.be/heubljqtJ0I?si=_WCKXqQx_xXpj5th

Overview

At Peblo, we build modern SaaS products that bring together collaborative workflows, AI-powered features, scalable backend systems, and intuitive frontend experiences. This challenge mirrors that reality in miniature.

Your task is to build a lightweight, collaborative, AI-powered notes workspace — a focused product that is small enough to complete on your own, yet rich enough to show how you think across the whole stack.

What this challenge evaluates

- Building cohesive full-stack applications
- Designing frontend and backend systems that work as one
- Structuring scalable APIs and services
- Integrating AI capabilities meaningfully — not as a gimmick
- Handling asynchronous workflows gracefully
- Thinking about product and system architecture

Assignment Objective

Build a full-stack application that delivers the following capabilities:

1. Lets users create and manage notes
2. Organises notes using tags or categories
3. Generates AI summaries from note content
4. Supports note search and filtering
5. Enables public sharing of notes
6. Displays simple productivity insights

System Requirements

Your system should implement the six components below. Each lists the expected behaviour, followed by an example data shape where useful.

1. Authentication

The system should support secure user authentication.

- User signup and login
- Protected routes and pages
- Persistent authentication sessions
- Secure password handling

EXAMPLE USER OBJECT

```
{
  "id": "USR_001",
  "name": "John Doe",
  "email": "john@example.com"
}
```

2. Notes Workspace

Users should be able to:

- Create and edit notes
- Auto-save note changes
- Organise notes using tags and categories
- Archive notes

EXAMPLE NOTE OBJECT

```
{
  "note_id": "NOTE_001",
  "title": "Project Planning",
  "tags": ["work", "meeting"],
  "updated_at": "2026-05-14T12:00:00Z"
}
```

3. AI Integration

Integrate an LLM or AI provider of your choice to generate:

- AI-generated summaries
- Action items extracted from notes
- Suggested titles from raw content

EXAMPLE EXPECTED OUTPUT

```
{
  "summary": "Weekly project planning discussion...",
  "action_items": ["Prepare UI mockups", "Review API structure"],
  "suggested_title": "Sprint Planning Notes"
}
```

4. Search & Filtering

Implement note discovery features such as:

- Keyword search
- Filtering by tags
- Sorting by most recently updated notes

The implementation may be simple, but it should feel responsive and usable.

5. Public Share Page

Users should be able to generate a public share link for a note.

- Shared notes accessible without login
- Proper handling of public vs. private visibility
- A clean, public-facing page

6. Productivity Insights

Create a simple dashboard that displays:

- Total notes
- Recently edited notes
- Most-used tags
- AI usage statistics
- A weekly activity summary

Suggested API Endpoints

You may structure your APIs differently — this is a reference, not a requirement. A typical design could include:

POST	/auth/signup
POST	/auth/login
GET	/notes
POST	/notes
PATCH	/notes/:id
POST	/notes/:id/generate-summary
GET	/shared/:shareId

Technology Choices

You are free to choose your stack. Pick tools you are productive in — we are far more interested in how you use them than which ones you pick. Some common options:

Layer	Examples
Frontend	React · Next.js · Vue
Backend	Node.js (Express / NestJS) · Python (FastAPI / Django)
Database	PostgreSQL · MongoDB · SQLite
AI Provider	OpenAI · Anthropic · Google Gemini · any equivalent service

You may use your own API keys during development. Never commit secrets, credentials, or API keys to your repository.

What We Evaluate

Your submission will be assessed across these areas:

Area	What we look for
Frontend Engineering	Component structure, state management, and UX quality
Backend Engineering	API structure and overall architecture
AI Integration	Effective, meaningful AI workflow integration
Database Design	Schema clarity and scalability
Code Quality	Readable, maintainable, well-organised code
Product Thinking	Overall cohesiveness of the application
Documentation	Clarity of setup instructions and explanation

Submission Requirements

Please submit the following four items.

1. GitHub Repository

Your repository should contain:

- Frontend source code
- Backend source code
- Setup instructions
- A README explaining your architecture
- An `.env.example` file

EXAMPLE .ENV.EXAMPLE

```
DATABASE_URL=  
JWT_SECRET=  
LLM_API_KEY=
```

Do not commit real credentials.

2. Setup Instructions

Your README should explain:

- How to install dependencies
- How to run the frontend and backend
- How to configure environment variables
- How to test the application

3. Demo Video

Provide a 5–10 minute walkthrough video showing:

- Authentication flow
- Notes workflow
- AI summary generation
- Search and filtering
- Public sharing flow
- Dashboard insights

This helps us evaluate your system quickly and see it the way you intended.

4. Sample Outputs

Include supporting samples such as:

- Example API responses
- AI-generated summaries
- Database schema or a sample dump
- Screenshots of the application

Optional — Nice to Have

These are not required, but they will be viewed positively if included:

Enhancement	Why it stands out
Realtime collaboration	Demonstrates handling of concurrent state
Markdown preview	Polish in the editing experience
Optimistic UI updates	Thoughtful, responsive frontend
Keyboard shortcuts	Attention to power-user UX
Dark mode	Care for visual detail
Deployment links	Shows you can ship to production
Automated tests	Confidence in code quality

Important Guidelines

Before you submit, please ensure:

- ✓ No secrets or API keys are committed to the repository
- ✓ Code is well structured and documented
- ✓ The application can run locally from a clean clone
- ✓ Frontend and backend are properly integrated
- ✓ AI-generated content is handled cleanly and predictably

We're excited to see what you build. Have fun with it.